When Detection Fails: Systematic Robustness Analysis of Contamination Detection Methods Across Contamination Types

Srishti Dutta, Debayush Dey, Arijit Samanta, Amar Sinha

Abstract

Benchmark contamination undermines the reliability of large language model (LLM) evaluation by inflating performance on benchmarks the model has effectively seen before. Prior work has proposed various detection methods. However their robustness to different forms of contamination remains unclear. Real-world contamination often appears in paraphrased or reformulated forms rather than exact duplicates. Thus raising the question of whether existing detectors generalize beyond surface-level overlap.

In this paper, we conduct a controlled robustness analysis of two widely used contamination detection methods: prefix completion and perturbation sensitivity across two contamination regimes: verbatim and paraphrased. We construct settings where the contamination type is known exactly using Phi-2-2.7B with LoRA fine-tuning on curated MMLU subsets. This enables direct comparison of detector behavior. We further evaluate an ensemble that combines both signals under a fixed false-positive budget.

Our results suggest that contamination detection is strongly contamination-type dependent. A simple accuracy-gap baseline is unreliable as a contamination signal. Prefix completion performs well only when the evaluation prompt aligns closely with the memorized surface form but it degrades under cross-type conditions. Perturbation sensitivity captures a complementary behavioral signal. It improves detection in some verbatim settings. It also exhibits reduced effectiveness under paraphrased contamination. The ensemble improves detection only when the two methods exhibit non-redundant failure modes and provides no gain when one signal dominates.

These findings suggest that contamination detection cannot be treated as a single-method or single-metric problem. Instead, detector performance must be evaluated across contamination types with attention to the underlying assumptions each method exploits. Our work highlights the need for robustness-oriented evaluation protocols and positions ensemble methods as diagnostic tools rather than universally reliable solutions.

Keywords: benchmark contamination, large language models, MMLU, prefix completion, perturbation sensitivity, robustness evaluation, contamination detection

1 Introduction

Benchmark contamination has become a central threat to trustworthy LLM evaluation. When a model has already encountered benchmark items, benchmark scores stop measuring generalization and begin reflecting memorization, recall, or exposure artifacts. This matters directly for model ranking, benchmark-based claims of capability, and any scientific conclusions drawn from benchmark comparisons.

The contamination problem is especially acute for public benchmarks such as MMLU and GSM8K. These benchmarks are widely circulated and discussed in tutorials and forums. They are often included indirectly in training corpora scraped from the web. In that setting, contamination is rarely a simple copy-paste event. A benchmark question may appear in a training corpus verbatim, but it may also appear in restated, summarized, translated, or lightly paraphrased form. If a detector is only calibrated on exact overlap, it may look reliable in the laboratory while failing in the real world [1, 2, 5, 6].

This is the gap our project addresses. The research question is:

How robust are existing contamination detection methods to different types of benchmark contamination, and can an ensemble approach improve detection?

We focus on two contamination types. In the case of **verbatim contamination**, the model is trained on the exact benchmark question-answer pairs, while in the case of **paraphrased contamination**, the model is trained on a lexically altered version that is semantically equivalent of the same example. These two settings hold semantic content constant while changing the surface form. Therefore providing a direct test of whether a detector is responding to memorized wording, memorized semantics or to some interaction between the two.

To study this question, we use a controlled experimental design instead of searching for unknown contamination in a frontier model. We have sampled 600 MMLU questions and organized them into three 200-example splits under verbatim, paraphrase-source, and clean. We fine-tune Phi-2-2.7B with LoRA on verbatim examples to obtain `Model_V`, and on paraphrased examples to obtain `Model_P`. We then evaluate two detectors: **prefix completion**, which tests whether a model can reconstruct a held-out suffix from partial prefixes, and **perturbation sensitivity**, which tests whether the model behaves brittlely under answer-preserving perturbations. Finally, we build an ensemble to test whether the two methods provide complementary signals.

The core contribution of the paper is not the invention of a new detector. It is a controlled robustness analysis of two existing detector families across contamination types. That distinction matters. The literature already suggests that contamination can evade exact matching and other lexical overlap methods, but there is much less controlled evidence about which contamination type defeats which detector, and why [1, 3, 4, 7]. Our experiments are intended to make those failure modes explicit.

We therefore focus on the controlled Phi-2 setting, where contamination type is known by construction and the failure modes of the detectors can be analyzed directly. The paper contributes a controlled evaluation framework, a comparison of two detector families with different assumptions, and an ensemble analysis that clarifies when combining them is actually useful.

2 Related Work

Recent work has made clear that contamination is broader than exact benchmark copying. Yang et al. argue in *Rethinking Benchmark and Contamination for Language Models with Rephrased Samples* that simple string-matching decontamination is insufficient because paraphrases and related variants can bypass overlap filters while still inflating performance [1]. That observation motivates our central design choice: robustness must be evaluated across contamination types, not only on exact duplicates.

Deng et al., in *Investigating Data Contamination in Modern Benchmarks for Large Language Models*, study contamination detection through retrieval-based methods and test-set guessing protocols, emphasizing contamination detection as a post-hoc inference problem rather than only a data-cleaning problem [2].

Dekoninck et al. introduce *ConStat: Performance-Based Contamination Detection in Large Language Models*, shifting the discussion from strict data overlap toward inflated, non-generalizing benchmark performance [3]. That perspective is important for our project because it suggests that contamination should be evaluated by behavioral signatures, not only by training-set provenance.

Most recently, Liang et al. propose *DVD: A Robust Method for Detecting Variant Contamination in Large Language Model Evaluation* [4]. DVD is especially relevant because it targets variant contamination directly and argues that common perplexity- and edit-distance-based approaches degrade on semantically preserved rewrites. Our work is complementary: rather than proposing a new detector, we compare two detector families already used in the literature and ask how their behavior changes when contamination shifts from verbatim to paraphrased.

Several additional works help position our study. Xu et al. benchmark leakage detection pipelines based on perplexity and n-gram accuracy [7]. Zhang et al. propose PaCoST, a paired-confidence significance test for benchmark contamination detection [8]. Survey work by Xu et al. and Ravaut et al. highlights the diversity of contamination definitions and detection assumptions now in use [5, 6]. Together, these papers reinforce the need to evaluate not only whether a detector works, but what kind of contamination it is actually sensitive to.

This positioning clarifies the novelty of our work. We do not claim to be the first to note that paraphrases are difficult. We instead provide a controlled comparison across contamination types using known ground truth, fine-tuned models, and side-by-side detector evaluation.

3 Experimental Setup

3.1 Contamination Types

We evaluate two contamination regimes.

Verbatim contamination is the case of direct leakage. The model is fine-tuned on the exact benchmark question, answer options, and label that later appear at the time of evaluation. This is the contamination setting that most existing lexical or prefix-based detectors implicitly assume.

Paraphrased contamination is the semantically matched but lexically altered case. The model is fine-tuned on a version of the benchmark item that is rewritten to preserve the answer and topic but changes wording and in some cases also the structure. This setting is more realistic because public benchmark content is most often discussed, summarized, or reformulated before ending up in a training corpus.

The core robustness test in this paper is the comparison between these two contamination types. We have held the semantics intentionally close while keeping surface form. If a detector's performance collapses under paraphrase, that detector is likely exploiting lexical familiarity rather than contamination in a broader sense.

Table 1: Conceptual distinction between the two contamination regimes.

Property	Verbatim contamination	Paraphrased contamination
Surface form match at evaluation	Exact	Different
Semantic content	Same	Same
Correct answer	Same	Same
Expected difficulty for lexical detectors	Lower	Higher
Realism in web-scale corpora	Moderate	High
Main failure risk	Overestimating detector strength	Missing contamination entirely

3.2 Dataset Construction

We construct 600 MMLU-derived examples divided into three 200-example splits:

Table 2: Dataset splits used in the controlled study.

Split	Size	Role
Verbatim	200	Exact examples used for verbatim contamination
Paraphrase source	200	Original examples later paraphrased for contamination
Clean	200	Nominally held-out examples used as negatives in detection

The sampling pipeline filters MMLU test questions by basic usability constraints: question length between 10 and 30 words, maximum length around 180 characters, and a valid four-option multiple-choice structure. The paraphrase-source pipeline additionally filters out fill-in-the-blank prompts and heavy numeric problems. The resulting sample is subject-diverse, although it is better described as balanced sampling than strict formal stratification. In the archived artifact, the split construction is not perfectly disjoint: the clean and verbatim splits share three examples. We therefore interpret the clean split as a nominally held-out negative set rather than a perfectly uncontaminated reference set.

For paraphrasing, we use an instruction-tuned model-based paraphrasing pipeline with sentence-transformer similarity checking. The pipeline targets cosine similarity above 0.90 using `all-MiniLM-L6-v2`, retries failed paraphrases with a lowered threshold schedule, and falls back to the original text when acceptable paraphrases are not obtained. The final log reports 189 successful paraphrases out of 200 and 11 fallbacks to original wording. As a result, the paraphrased split is best described as a high-similarity paraphrase set with a small amount of residual verbatim carryover.

That imperfection is a limitation, but it is also realistic: real contamination corpora are not neatly partitioned into fully paraphrased and fully verbatim cases.

Table 3: Representative example from the paraphrase pipeline.

Version	Question text
Original	"Which of the following best accounts for the negative slope of the liquid-solid equilibrium line in the phase diagram for water?"
Paraphrased	"Which of the following best explains the negative slope of the liquid-solid equilibrium line in the phase diagram for water?"

Even this simple example illustrates an important point. The semantic content is unchanged, the answer remains unchanged, and the lexical changes are light but real. For detectors based on exact textual continuation, such small changes can still matter.

3.3 Controlled Model Construction

We use Microsoft Phi-2-2.7B as the base model and perform LoRA fine-tuning to create two controlled models:

Table 4: Controlled models used in the study.

Model	Training split	Intended contamination type
`Model_V`	Verbatim split	Verbatim contamination
`Model_P`	Paraphrased split	Paraphrased contamination

Training uses 8-bit loading, LoRA with rank 16 and alpha 32, and formatted multiple-choice prompts ending in the answer letter. Fine-tuning runs for 20 epochs with batch size 2, gradient accumulation 8, learning rate `2e-4`, and sequence length 256.

The study plan originally targeted memorization verification above 90% accuracy on the contaminated split for both models. The finalized experiment does meet that criterion, but the accuracy differences, while present, are insufficiently reliable to serve as a standalone contamination signal. This is because they reflect a mixture of memorization, distributional shift between splits, and generalization effects rather than contamination alone. This sharply limits the value of any accuracy-gap interpretation and is one reason we treat the detector results as exploratory evidence about robustness rather than definitive proof of strong memorization.

3.4 Detection Methods

3.4.1 Accuracy-gap baseline

The simplest baseline compares accuracy on contaminated versus clean data:

$$\text{Contamination Gap} = \text{Accuracy(contaminated split)} - \text{Accuracy(clean split)}$$

In principle, a contaminated model should perform better on examples it has effectively seen before. In practice, this baseline is only meaningful if fine-tuning produces a large and stable memorization effect at the answer level.

3.4.2 Prefix completion

The prefix-completion detector tests whether the model can continue a question from partial text in a way that matches the original suffix. For each example, the detector uses 25%, 50%, and 75% of the question as the prefix, asks the model to complete the remainder, and scores the generated suffix

against the gold suffix. The project materials describe ROUGE-L and normalized edit distance as the main similarity components, aggregated across the three prefix lengths into one contamination score per example.

The intuition is straightforward: if a model has memorized a benchmark item in a surface-form-specific way, it should complete that item more faithfully than it completes a clean example.

3.4.3 Perturbation sensitivity

The perturbation detector measures how much the model's behavior changes when answer-preserving variants of an example are applied. The original team plan specified four perturbations: name swap, number change, option shuffle, and minor paraphrase. The final improved pipeline in the repository uses a safer answer-preserving set: name swap, option shuffle, minor paraphrase, and prompt wrapping, explicitly replacing noisier perturbations that could accidentally change the task.

For each example, the model is scored on the original item and on its perturbations. The detector treats a large drop in correct-answer confidence or stability under perturbation as a memorization signature: the model appears tied to one phrasing or ordering rather than using a robust underlying solution process.

3.4.4 Ensemble detector

The ensemble combines normalized prefix and perturbation scores. In the improved pipeline, weights are tuned on a held-out validation split and evaluated on a disjoint test split under a fixed false-positive budget. The main quantity of interest is:

$$\Delta\text{TPR} = \text{TPR}(\text{ensemble}) - \max(\text{TPR}(\text{prefix}), \text{TPR}(\text{perturbation}))$$

This formulation is useful because it asks whether the ensemble actually contributes beyond the better individual detector.

More formally, if $s_{\text{prefix}}(x)$ and $s_{\text{pert}}(x)$ denote normalized detector scores for example x, the ensemble score is

$$s_{\text{ens}}(x) = \alpha\, s_{\text{prefix}}(x) + (1 - \alpha)\, s_{\text{pert}}(x)$$

where α is selected on a held-out validation split. In the improved pipeline, weight selection is constrained by a false-positive budget rather than by AUC alone. That choice reflects a practical point: a detector with an attractive AUC can still be unusable if the threshold needed to realize that performance yields too many false alarms.

3.5 Evaluation Protocol

The evaluation protocol differs slightly across the two detector families, but the overall logic is consistent. For each contamination setting, the contaminated samples are considered positive cases, while the clean samples are seen as negative cases. This changes the task into a binary detection problem instead of a regular question-answering task. Since the archived clean split overlaps slightly with the verbatim split, we should view the evaluation results as rough indicators rather than completely accurate detection benchmarks.

In the prefix completion approach, the detector generates a single score for each example by measuring how similar the model's generated continuation is to the actual gold suffix. These scores then become binary predictions using various threshold values, and ROC curves are created by changing the threshold across possible values. From these curves, we report metrics such as ROC-AUC, true positive rate (TPR), false positive rate (FPR), and the threshold selected in the original Member 2 pipeline.

For perturbation sensitivity, the detector gives a sensitivity score to each example by examining how much the model's behavior changes between the original input and its altered versions. In the improved pipeline, the scoring method emphasizes drops in correct-answer probability instead of relying only on whether the answer changes completely. This makes the evaluation more dependable and informative, especially for borderline cases. The final ensemble normalizes prefix and perturbation scores, selects weights on a held-out validation portion, and evaluates performance on the remaining test portion under a fixed false-positive budget. This behavioral framing is closer in spirit to performance-based methods such as ConStat and confidence-based tests such as PaCoST than to pure overlap-based decontamination [3, 8].

The study uses TPR degradation across contamination types as one of its main robustness diagnostics. That metric is important because a detector can achieve a respectable average AUC while still collapsing on the contamination type that matters most in practice. In other words, robustness is not captured by absolute performance alone; it is captured by how much performance changes when the contamination mechanism changes.

3.6 Statistical Analysis

For a detector score $s(x)$ and threshold τ, an example x is classified as contaminated when $s(x) \geq \tau$. The standard operating-point metrics are then

$$\mathrm{TPR} = \frac{TP}{TP + FN} \qquad \text{and} \qquad \mathrm{FPR} = \frac{FP}{FP + TN}.$$

ROC-AUC is used as the threshold-free ranking metric. In the ensemble experiments, we additionally report ΔTPR, defined as the gain of the ensemble over the best individual detector at the selected

operating point. Where available, the improved pipeline also reports bootstrap confidence intervals and paired t-tests. These statistics do not remove uncertainty, but they do help distinguish genuine complementarity from small changes caused by thresholding or finite sample size.

4 Results

4.1 Accuracy-gap baseline

The accuracy results are:

Table 5: Accuracy-gap baseline results.

Split	Accuracy
Model_P on Paraphrased	0.990
Model_P on Verbatim	0.875
Model_P on Clean	0.525

The contamination gap here, as we can see, is present but moderate.

This is an informative result. The controlled fine-tuning setup shows clear performance differences across input types. Trained on paraphrased data, the model achieves near-perfect accuracy on paraphrased inputs (0.990), but drops on verbatim inputs (0.875), indicating sensitivity to surface-form changes despite semantic similarity, and declines further on the clean dataset (0.525), reflecting a broader distribution shift.

The smaller drop to verbatim inputs indicates partial robustness to rewording, while the larger drop on the clean split highlights limited generalization to unseen data.

Overall, performance differences reflect both memorization of training-style patterns and sensitivity to distributional changes. Thus, accuracy alone should be interpreted cautiously as a contamination signal, as it may combine memorization effects with generalization gaps.

Table 6: High-level summary of detector behavior.

Method	Main positive result	Main weakness
Accuracy gap	Moderate in this setup	Accuracy mixes memorization and generalization, so the cause is unclear
Prefix completion	Strong when evaluation wording matches memorized wording	Weak cross-type transfer
Perturbation sensitivity	Adds complementary signal for verbatim contamination	Degrades on paraphrased contamination
Ensemble	Improves TPR for verbatim contamination	Little gain when one component dominates

4.2 Prefix completion is highly contamination-type dependent

The prefix-completion results are:

Table 7: Prefix-completion detection results.

Condition	ROC-AUC	TPR	FPR
`Model_V` on verbatim	0.707	0.695	0.365
`Model_V` on paraphrased	0.525	0.475	0.375
`Model_P` on verbatim	0.464	0.030	0.005
`Model_P` on paraphrased	0.801	0.725	0.205

Two patterns stand out.

First, prefix completion works only when the memorized wording and the evaluated wording are aligned enough. `Model_V` is reasonably detectable on verbatim items, and `Model_P` is strongly detectable on paraphrased items. In both cases, the model is being tested against the same general form it saw during fine-tuning.

Second, cross-type transfer is poor. `Model_V` tested on original wording but evaluated through paraphrased contamination is near random with AUC 0.525. `Model_P` tested on verbatim originals is even worse, with AUC below 0.5 and TPR nearly zero. This also changes the earlier assumption that paraphrased contamination would naturally be harder for prefix completion to detect than verbatim contamination. In our experiments, `Model_P` actually performed best on the paraphrased examples among all four conditions. This suggests that the detector is not only reacting to exact copied text, but also to certain writing patterns and phrasing styles that remain similar even after rewording. The results indicate that detection performance can vary depending on how the paraphrases were created, how the model was fine-tuned, and how closely the training and testing language patterns match each other.

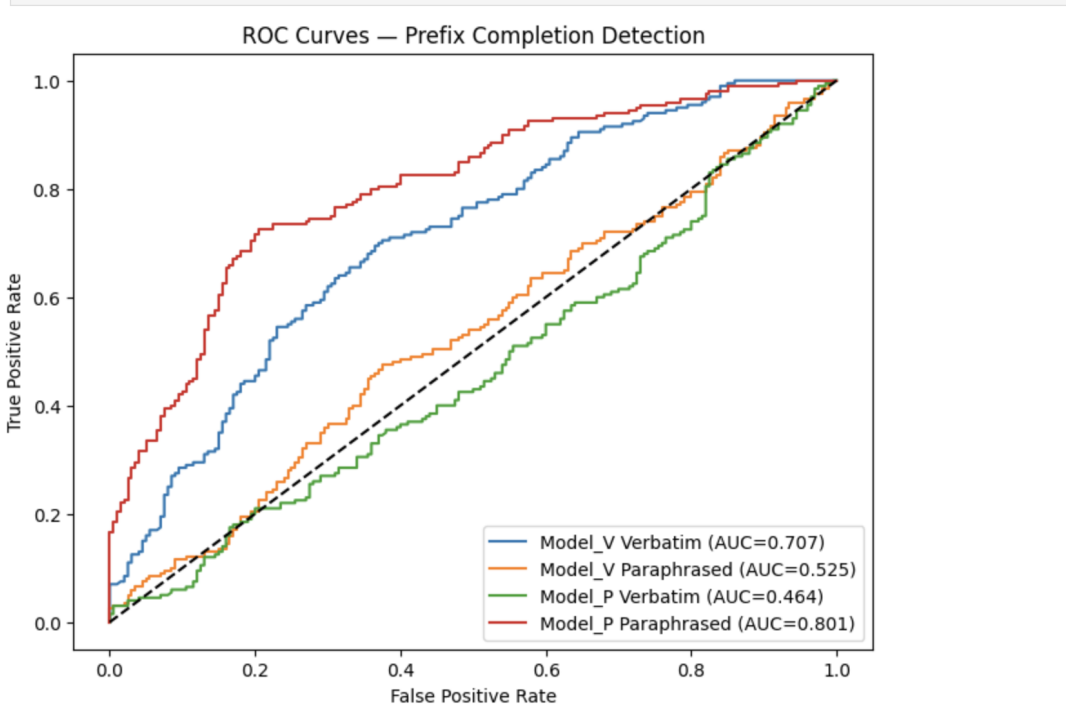


Figure 1: ROC behavior of the prefix-completion detector across contamination settings.

4.3 Perturbation sensitivity adds a complementary signal

The perturbation and ensemble results reported here come from the improved-batch pipeline used in the last stage of the study. This pipeline replaces noisier perturbations with safer answer-preserving transformations and tunes ensemble thresholds under a fixed false-positive budget. These results represent the final archived summary of the Member 3 pipeline, although the corresponding raw outputs were not packaged as cleanly as they should be for a polished release.

The final improved-batch comparison is:

Table 8: Perturbation and ensemble results from the improved-batch pipeline.

Condition p-value	Prefix TPR	Prefix FPR	Prefix AUC	Perturb TPR	Perturb FPR	Perturb AUC	Ensemble TPR	Ensemble FPR	Ensemble AUC	Ensemble weights	ΔTPR
`model_v_verbatim` 0.0034	0.42	0.14	0.7259	0.49	0.35	0.6750	0.64	0.33	0.7653	prefix 0.55 / perturb 0.45	+0.15
`model_p_paraphrased` 1.0000	0.68	0.14	0.8104	0.35	0.25	0.6058	0.68	0.15	0.8139	prefix 0.95 / perturb 0.05	0.00

These results support a more nuanced conclusion than either method alone would suggest, while still needing to be read in light of the study's data and packaging limitations.

For `model_v_verbatim`, the perturbation sensitivity is slightly stronger than prefix completion in TPR, even though FPR also increased. The ensemble improves TPR to 0.64, resulting in a +0.15 gain over the best individual detector and a paired t-test p-value of 0.0034 indicating that the result

is statistically significant. This is the strongest positive ensemble result in our project and suggests that the two detectors produce complementary signals under verbatim contamination.

For `model_p_paraphrased` however prefix completion remains the best practical detector. Perturbation sensitivity drops significantly, and the ensemble assigns it almost no weight. As a result the ensemble collapses back to the prefix detector, producing no meaningful TPR gain. This tells us that the ensemble is not robust for all contamination types but only when the detectors fail in a complementary manner.

The project summary also reports degradation values across contamination types:

- Prefix degradation: -0.26
- Perturbation degradation: $+0.14$
- Ensemble degradation: -0.04

The sign convention here implies that perturbation sensitivity performs better on verbatim contamination than on paraphrased contamination, while prefix completion in this setup is actually stronger on paraphrased contamination than verbatim contamination. Again, this should be treated as an empirical outcome of the archived setup rather than forced into a pre-registered expectation.

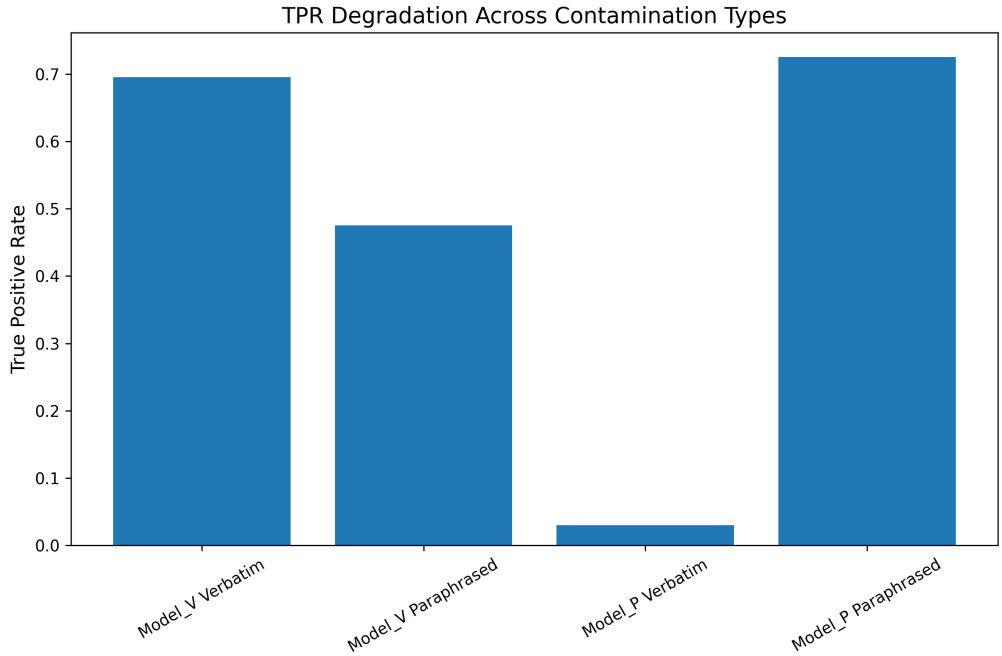


Figure 2: Degradation in true positive rate as contamination shifts from verbatim to paraphrased.

4.4 Failure modes

The detectors fail for different reasons.

Prefix completion fails when lexical anchoring is weak. If the memorized training item differs sufficiently in wording from the evaluation prefix, the model may know the underlying content without reproducing the expected suffix. In that regime, low completion overlap does not mean no contamination; it may only mean that contamination lives at the semantic or answer level rather than the exact text level.

Perturbation sensitivity fails when the perturbations are either too weak or too noisy. Weak perturbations produce little signal because both contaminated and clean examples remain stable. Noisy perturbations can change the task itself and produce false positives. The project's improved perturbation pipeline is notable precisely because it moved away from noisier perturbations such as arbitrary number changes and toward safer answer-preserving transformations.

The ensemble fails when one component dominates. On paraphrased contamination, the optimized ensemble gives 95% of the weight to prefix scores. In that setting, there is no meaningful complementarity left to exploit, so the ensemble cannot improve TPR. Ensembles are only as useful as the diversity of the errors they combine.

Taken together, these failure modes support the paper's main claim that contamination detection should be evaluated as a robustness problem, not merely as a leader board problem.

One useful way to read the results is through the relationship between memorization and representation. Prefix completion is strongest when contamination leaves a reconstructible textual trace, whereas perturbation sensitivity is strongest when contamination creates fragile behavior under small semantic-preserving edits. These are related but distinct signatures of contamination, and the ensemble helps only when both are present and non-redundant.

5 Discussion

The above study highlights a few crucial observations. In our experiment the accuracy gaps were difficult to interpret as they combined not only generalization but also sensitivity to wording changes to the memorization signal. We also see that the detector performance depended heavily on the type of contamination and while some methods worked well for verbatim contamination they became less reliable for paraphrased contamination. The ensemble results were also mixed. While combining detectors improved performance in the verbatim setting the same does not happen for paraphrased setting, thus suggesting that ensembles are only useful when detectors produce complementary signals to each other.

A useful next step would be to perform a case study on open weight models like LLaMA-2-7B and

Mistral-7B on MMLU and GSM8K and see if the trends and parameters observed in the current setup remain stable.

It is also worth emphasizing what this study does and does not say about contamination in the wild. The controlled setting is not intended to estimate the contamination rate of any deployed model. Instead, it functions as an instrumented environment for asking causal questions. If a detector succeeds or fails, we know exactly what kind of contamination it is responding to.

This matters for methodology. Much of the contamination literature mixes two questions that should be separated: Is benchmark X contaminated in model Y? And what property of a detection method makes it robust or brittle? Our study is aimed primarily at the second question.

The results also have a practical implication for benchmark governance. If benchmark maintainers evaluate contamination detectors only against verbatim overlap or near-duplicate continuation, they may systematically overestimate detector reliability. Any credible contamination evaluation protocol should include paraphrased or variant contamination as a standard stress test [1, 3, 4].

There is also a reporting implication. A paper that introduces a contamination detector should not report a single pooled AUC in isolation. It should specify the contamination regime, the perturbation or rewriting process, the operating threshold, and whether calibration was done globally or per condition.

6 Limitations and Threats to Validity

This project has several limitations, and stating them clearly strengthens rather than weakens the paper.

Incomplete realization of the original plan. The finalized experiments do not meet the planned 90% memorization verification target, and the broader real-model case study is deferred. The paper should therefore be read as a controlled robustness study rather than a full contamination audit of public models.

Experiment packaging remains imperfect. Intermediate outputs from the perturbation and ensemble pipeline were not consolidated cleanly into one finalized artifact.

Paraphrase purity is imperfect. The paraphrase notebook reports 189 successful paraphrases and 11 fallbacks to original text. This means the paraphrased contamination split contains a small amount of verbatim residue. That likely reduces the clean separation between contamination types.

Dataset scale is modest. With 200 examples per split, the study is large enough to reveal directional effects but still small relative to the space of possible MMLU subjects and linguistic transformations.

Model scope is narrow. All controlled experiments are built around Phi-2-2.7B with LoRA fine-

tuning. The observed detector behavior may not transfer quantitatively to larger instruction-tuned models or to very different domains such as code generation.

The paraphrase generator is itself a modeling choice. Paraphrases with high-similarity that are generated by an instruction-tuned model are only one form of contamination. Other forms like statements rewritten by humans or translated rewrites, could affect detector behavior differently.

Threshold selection may be condition-specific. The thresholds and ensemble weights were tuned for the current experimental setup, but they may not generalize to other benchmarks or models without retuning them.

The current controlled setup does not all possible contamination scenarios. In real world, contamination can come from sources like pretraining datasets, synthetic instruction data, translated material, benchmark discussions, or published solutions.

Possible future work includes improving the paraphrase split, consolidate the perturbation outputs, and run a case study on open weight models using the current thresholds to see if the results generalize to other models and benchmarks.

7 Conclusion

This paper asked whether contamination detectors are robust across contamination types. The answer is no, at least not by default.

In a controlled MMLU-based study with Phi-2 fine-tuned on verbatim and paraphrased contamination, we find that a simple accuracy-gap baseline is not a reliable indicator, prefix completion is strongly dependent on surface-form alignment, perturbation sensitivity contributes a complementary but uneven signal, and an ensemble improves detection only in the condition where the two methods fail differently enough to help each other.

Thus the main contribution here is a robustness study. The contamination detectors should not be judged by a single combined score or by their performance on verbatim contamination alone. They must be tested across all types of contamination with the known ground truth.

The main takeaway is that contamination detector performance varied significantly across different contamination types. That is exactly why controlled comparisons and ensemble diagnostics are necessary.

References

[1] Shuo Yang, Wei-Lin Chiang, Lianmin Zheng, Joseph E. Gonzalez, and Ion Stoica. Rethinking Benchmark and Contamination for Language Models with Rephrased Samples. *CoRR*,

abs/2311.04850, 2023. https://doi.org/10.48550/arXiv.2311.04850

[2] Chunyuan Deng, Yilun Zhao, Xiangru Tang, Mark Gerstein, and Arman Cohan. Investigating Data Contamination in Modern Benchmarks for Large Language Models. *CoRR*, abs/2311.09783, 2023. https://doi.org/10.48550/arXiv.2311.09783

[3] Jasper Dekoninck, Mark Niklas Mueller, and Martin Vechev. ConStat: Performance-Based Contamination Detection in Large Language Models. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024. https://openreview.net/forum?id=ALISPmDPCq

[4] Renzhao Liang, Jingru Chen, Bo Jia, Bo Deng, Chenggang Xie, Yidong Wang, Ke Jin, Xin Wang, Linfeng Zhang, and Cunxiang Wang. DVD: A Robust Method for Detecting Variant Contamination in Large Language Model Evaluation. ACL ARR 2026 January Submission, 2026. https://openreview.net/forum?id=waELCVXKTP

[5] Cheng Xu, Shuhao Guan, Derek Greene, and M. Tahar Kechadi. Benchmark Data Contamination of Large Language Models: A Survey. *CoRR*, abs/2406.04244, 2024. https://doi.org/10.48550/arXiv.2406.04244

[6] Mathieu Ravaut, Bosheng Ding, Fangkai Jiao, Hailin Chen, Xingxuan Li, Ruochen Zhao, Chengwei Qin, Caiming Xiong, and Shafiq Joty. How Much are LLMs Contaminated? A Comprehensive Survey and the LLMSanitize Library. *CoRR*, abs/2404.00699, 2024. https://doi.org/10.48550/arXiv.2404.00699

[7] Ruijie Xu, Zengzhi Wang, Run-Ze Fan, and Pengfei Liu. Benchmarking Benchmark Leakage in Large Language Models. *CoRR*, abs/2404.18824, 2024. https://doi.org/10.48550/arXiv.2404.18824

[8] Huixuan Zhang, Yun Lin, and Xiaojun Wan. PaCoST: Paired Confidence Significance Testing for Benchmark Contamination Detection in Large Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 1794–1809, 2024. https://doi.org/10.18653/v1/2024.findings-emnlp.97